DAA Lab - Experiment 4

Implement the Fractional Knapsack Problem using Greedy Algorithm

Dr. Ayush Kumar Agrawal
SoCS
UPES

Objective

- Solve the Fractional Knapsack Problem using the Greedy approach.
- Maximize profit by selecting items based on value-to-weight ratio.
- Understand greedy choice property and optimal substructure.

Theory

- Knapsack problem: choose items to maximize profit without exceeding capacity.
- Fractional version: items can be broken into fractions.
- Greedy approach:
  1. Compute ratio = value/weight for each item.
  2. Sort items in descending order of ratio.
  3. Pick items fully until capacity allows.
  4. If remaining capacity is less, pick fraction of next item.

Algorithm: Fractional Knapsack (Greedy)

Fractional_Knapsack(W, n, value[], weight[])

1. For each item i: ratio[i] = value[i]/weight[i]
2. Sort items in descending order of ratio
3. profit = 0, capacity = W
4. For i = 1 to n:
   - If weight[i] $\leq$ capacity:
     - profit += value[i]
     - capacity -= weight[i]
   - Else:
     - profit += ratio[i] * capacity
     - break
5. Return profit

C Code - Fractional Knapsack (For Reference)

```
#include <stdio.h>

void fractionalKnapsack(int n, float W,
                        float weight[], float value[]) {
    float ratio[20], temp;
    float total = 0;
    int i, j;

    // compute ratio
    for (i=0; i<n; i++)
        ratio[i] = value[i] / weight[i];

    // sort by ratio (descending)
    for (i=0; i<n-1; i++) {
        for (j=i+1; j<n; j++) {
            if (ratio[i] < ratio[j]) {
                temp = ratio[i]; ratio[i] = ratio[j]; ratio[j] = temp;
                temp = weight[i]; weight[i] = weight[j]; weight[j] = temp;
                temp = value[i]; value[i] = value[j]; value[j] = temp;
            }
        }
    }
```

Cntd.)

```
    // fill knapsack
    for (i=0; i<n; i++) {
        if (weight[i] <= W) {
            total += value[i];
            W -= weight[i];
        } else {
            total += ratio[i] * W;
            break;
        }
    }

    printf("Maximum Profit = %.2f\n", total);
}
```

Cntd.)

```
int main() {
    int n, i;
    float W;

    printf("Enter number of items: ");
    scanf("%d", &n);
    float weight[n], value[n];

    printf("Enter weights: ");
    for (i=0; i<n; i++) scanf("%f", &weight[i]);
    printf("Enter values: ");
    for (i=0; i<n; i++) scanf("%f", &value[i]);

    printf("Enter knapsack capacity: ");
    scanf("%f", &W);

    fractionalKnapsack(n, W, weight, value);
    return 0;
}
```

Sample Input/Output

Input:
n = 3
Weights = 10, 20, 30
Values = 60, 100, 120
Capacity = 50
Output:
Maximum Profit = 240.00

Complexity Analysis

- Sorting: $O(n^2)$ (in this code)
- Greedy selection: $O(n)$
- Overall: $O(n^2)$, but with better sorting (like quicksort) $\rightarrow O(n \log n)$
- Space complexity: $O(n)$

Observation Table

Items	Capacity	Max Profit
3	50	240
4	60	280
5	40	200

Comparison with 0/1 Knapsack

- Fractional knapsack: items can be broken into parts, greedy works.
- 0/1 knapsack: items cannot be divided, needs dynamic programming.
- Fractional knapsack is simpler and faster.

Viva Questions

1. Why does greedy approach work for fractional knapsack but not 0/1 knapsack?
2. What is the time complexity of fractional knapsack?
3. Explain the role of ratio (value/weight).
4. Give a real-world application of fractional knapsack.

Conclusion

- Greedy algorithm works optimally for fractional knapsack.
- Sorting by ratio ensures maximum profit.
- Complexity mainly depends on sorting step.

References

- Cormen et al., *Introduction to Algorithms*.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.